Nadar Saraswathi Group of Institutions (TMHNU)

TMS GPS Telemetry & Real-Time Tracking Architecture Guide

Version: 2.4 (Enterprise Edition)
Date: October 2026
Target Port: TCP 5050 / HTTP Webhook

> **Overview & Purpose:** This document explains how physical GPS tracking devices installed on college buses transmit telemetry data, how your backend receives and decodes raw socket packets, how coordinates are verified and stored in MySQL, and how the frontend displays live buses on the map.

1. End-to-End Telemetry Pipeline

High-Speed GPS Telemetry Flow: From Bus Hardware to Live Administrative Map



2. Hardware Configuration: What to Give the GPS Vendor

Provide the following exact configuration parameters to your GPS hardware vendor or auto-electrician when programming the bus tracker devices:

Configuration Parameter	Recommended Setting	Technical Explanation
Server Address / Host	`<YOUR_PUBLIC_IP>` or `tms.college.edu`	Public static IP or domain name pointing to your backend server host.
Connection Protocol	`TCP Socket` (Stream)	Reliable connection-oriented transport protocol ensuring zero packet loss.
Target Port	`5050`	Configured in `server/services/gpsReceiver.js` (Default GPS port).
Moving Update Interval	`5 to 10 seconds`	Frequency when vehicle ignition is ON and moving along highway/city roads.
Stationary Interval	`30 to 60 seconds`	Frequency when bus is parked or idling to conserve device battery and cellular data.
Fallback Protocol	`HTTP POST Webhook`	Optional fallback at `http://<IP>:5001/api/tracking/webhook` if TCP is unsupported.

Cloud VPS Hosting (AWS / DigitalOcean)

1. Assign an Elastic Public IP to your server instance.
2. Open **Inbound Security Group / Firewall** for TCP Port `5050` from Any (`0.0.0.0/0`).
3. Ensure web port `5001` / `443` is also open for API & Web tracking.

Local College On-Premises Server

1. Obtain a Public Static IP from your Internet Service Provider (BSNL/Airtel).
2. In your campus gateway router, add a **Port Forwarding** rule:

External Port `5050` → Internal Local IP (e.g. `192.168.1.100:5050`).

Telemetry Protocol & Mathematical Decoding

Section 3 & 4: Packet Structure, NMEA Conversions & Socket Handling

Protocol: Standard ASCII Frame
Delimiter: Starts with @@, Ends with #

3. Telemetry Packet Structure

GPS tracking units send structured comma-delimited ASCII strings enclosed by @@ and #:

```
@@863071010318308,,,094159,1000.5210,N,07728.8400,E,28.10,191026,A,0101,1,0,25338,00030,#
```

Index	Field Name	Sample Value	Description & Verification Rules
[0]	Device IMEI	863071010318308	15-digit unique hardware identity mapped to buses.gps_device_id.
[3]	UTC Time	094159	Time in HHMMSS UTC format (09:41:59 UTC = 15:11:59 IST).
[4]	Raw Latitude	1000.5210	NMEA Latitude string: Degrees (10) + Minutes (00.5210).
[5]	Lat Hemisphere	N	North ('N') or South ('S'). South coordinates result in negative values.
[6]	Raw Longitude	07728.8400	NMEA Longitude string: Degrees (077) + Minutes (28.8400).
[7]	Lon Hemisphere	E	East ('E') or West ('W'). West coordinates result in negative values.
[8]	Speed in Knots	28.10	Ground speed measured in nautical knots (1 knot = 1.852 km/h).
[9]	UTC Date	191026	Date in DDMMYY format (19th October 2026).
[10]	Fix Status	A	**'A'** = Valid Satellite Fix. **'V'/'D'** = Invalid / Dead reckoning (Rejected).
[11]	I/O Status	0101	Bit 0 = Ignition (1=ON, 0=OFF), Bit 1 = Door Sensor (1=Open, 0=Closed).

4. Mathematical Coordinate & Unit Conversions

📍 NMEA to Decimal Degrees

GPS units output in DDMM.MMMM format. Decimal degrees are calculated by separating degrees and dividing minutes by 60:

```
Decimal = Degrees + (Minutes / 60)
```

Worked Example (Latitude 1000.5210 N):
Degrees = 10, Minutes = 00.5210
10 + (0.5210 / 60) = **10.008683° N**

Worked Example (Longitude 07728.8400 E):
Degrees = 77, Minutes = 28.8400
77 + (28.8400 / 60) = **77.480667° E**

⚡ Knots to Km/h Conversion

Speed is reported in maritime knots. Multiply knots by standard conversion factor 1.852:

```
Speed (km/h) = Speed (knots) × 1.852
```

Worked Example:
Raw speed = 28.10 knots
28.10 × 1.852 = **52.04 km/h** (Normal bus cruising speed)

🛡️ Trust Boundary Validator

If Fix Status is 'V' (Void) or 'D' (Dead reckoning), the backend drops the coordinate to prevent faulty teleportation glitches on the map.

5. Backend TCP Socket Implementation (`gpsReceiver.js`)

```
// server/services/gpsReceiver.js
const net = require('net');
const server = net.createServer((socket) => {
  let buffer = '';
  socket.on('data', async (chunk) => {
    buffer += chunk.toString('ascii');
    while (buffer.includes('#')) {
      const hashIdx = buffer.indexOf('#');
      const packet = buffer.substring(0, hashIdx + 1).trim();
      buffer = buffer.substring(hashIdx + 1);

      if (packet.startsWith('@')) {
        const parsed = parseGpsPacket(packet);
        if (parsed && parsed.valid) {
          await processTelemetry(parsed); // Writes to DB and emits live SSE
        }
      }
    }
  });
});
server.listen(5050, '0.0.0.0');
```

Database Storage, Geofencing & UI Rendering

Section 6, 7 & 8: MySQL Schema, Haversine Distance & Frontend Map

Storage: MySQL InnoDB
Frontend: React + MapLibre GL

6. Database Schema & Relational Storage

Incoming GPS fixes are associated with the bus registry and saved to the high-throughput `bus_locations` historical telemetry table:

```
-- 1. Bus registry maps IMEI to the vehicle
SELECT b.id, b.registration_number, b.bus_code, b.route_id, r.route_name
FROM buses b LEFT JOIN routes r ON r.id = b.route_id
WHERE b.gps_device_id = '863071010318308' LIMIT 1;

-- 2. Telemetry insert into bus_locations table
INSERT INTO bus_locations (bus_id, trip_id, imei, latitude, longitude, speed_km, ignition, recorded_at, raw_packet)
VALUES (14, 102, '863071010318308', 10.008683, 77.480667, 52.0, 1, '2026-10-23 09:41:59', '@@863071010318308...#');
```

7. College Campus Geofencing (Haversine Distance Engine)

The system actively measures the distance of each bus to the college campus centers using spherical trigonometry:

$$d = 2R \times \arcsin\left(\sqrt{\left[\sin^2\left(\frac{\Delta lat}{2}\right) + \cos(lat1) \times \cos(lat2) \times \sin^2\left(\frac{\Delta lon}{2}\right)\right]}\right)$$

Campus Name	Center Coordinates [Lon, Lat]	Geofence Radius	Automated Status Trigger
Vadapudupatti Main Campus	`[77.5082, 10.0427]`	`1.5` km radius	Triggers **"Inside Main Campus"** when distance ≤ 1.50 km.
Theni City Campus	`[77.4768, 10.0104]`	`1.0` km radius	Triggers **"Inside City Schools Campus"** when distance ≤ 1.00 km.

8. Real-Time Frontend Map Rendering (`LiveTracking.jsx`)

The browser never needs to refresh or poll aggressively. Instead, it subscribes to Server-Sent Events (SSE):

📡 Server-Sent Events (SSE) Stream

Client connects to `/api/tracking/stream` once. Whenever the GPS socket receives a coordinate, the server pushes an instantaneous telemetry JSON event to all connected admin and parent screens.

🗺️ MapLibre GL Polygon & Pin Animation

Renders `<MapGeoJSON>` for the translucent campus radius and calculates bearing angles to smoothly rotate bus icons in the direction of road travel.

9. Testing & Troubleshooting Checklist

Symptom / Scenario	Root Cause	Resolution Steps
TCP Port 5050 EADDRINUSE	Another Node or service process is holding port 5050.	Run `Get-Process -Id (Get-NetTCPConnection -LocalPort 5050).OwningProcess | Stop-Process`.
Bus Marker Not Appearing	IMEI is not mapped in the `buses` database table.	Go to Bus Management → Edit Bus → Enter device IMEI into `gps_device_id` field.
Coordinates jumping on map	GPS satellite antenna obstructed (Fix status 'V'/'D').	Ensure GPS antenna has clear line of sight to open sky; validator will auto-reject bad fixes.
Manual Test Packet Injection	Verifying connection from command line without bus.	Run in PowerShell: `$c=New-Object System.Net.Sockets.TcpClient("localhost",5050); $s=$c.GetStream(); $w=New-Object System.IO.StreamWriter($s); $w.WriteLine("@@863071010318308,,,094159,1000.5210,N,07728.8400,E,28.10,191026,A,0101#"); $w.Flush(); $c.Close()`.